Learning to Navigate a Mobile Robot in Randomized Environments with Deep Deterministic Policy Gradients and a Frontier-Style Exploration Heuristic

Nikita Sviridov
nikswirlo@gmail.com

July 4, 2026

Abstract

We study autonomous point-to-point navigation of a mobile robot in bounded planar fields cluttered with rectangular obstacles. Unlike a fixed-map formulation, every training episode samples a fresh task: the number, sizes and positions of the obstacles and the robot's start pose are all drawn at random, and a breadth-first search over an inflated occupancy grid certifies that each sampled layout admits a collision-free corridor to the goal. The robot perceives its surroundings through a simulated 180° lidar and is driven by a continuous throttle/steering command, which makes the control problem a natural fit for actor–critic reinforcement learning. We formulate the task as a Markov decision process, solve it with *Deep Deterministic Policy Gradient* (DDPG), and augment the agent with a frontier-style *point-of-interest* (POI) heuristic that turns raw lidar returns into a compact, goal-directed exploration signal. Trained for 3000 episodes in minutes on consumer Apple-Silicon hardware, the deterministic policy reaches the goal on 74.7% of 300 previously unseen random layouts. Because the map changes every episode, this figure measures generalization across tasks rather than the memorization of a single route.

1 Introduction

Reinforcement learning (RL) has become a standard tool for continuous control, where the agent must output real-valued commands rather than choose from a discrete menu of actions [4]. Mobile-robot navigation is a canonical instance: the robot continuously sets its speed and heading to reach a goal while avoiding collisions, using only local range measurements. Value-based methods such as DQN [3] do not apply directly because the action space is continuous; deterministic policy gradients [2] and their deep counterpart DDPG [1] close this gap by learning a parametric actor jointly with a critic.

A policy trained on a single fixed map easily overfits: it can memorize one route instead of learning to navigate. We therefore train with *domain randomization* of the layout — every episode presents a new obstacle configuration and start pose — so the only strategy that survives is one that reads the lidar and the sub-goal features and reacts to the map it is actually in.

This report documents an end-to-end navigation system built around DDPG. Our contributions are: (i) a Markov-decision-process formulation of a lidar-driven navigation environment (*ChopperScape*) with per-episode layout randomization and a BFS-certified solvability guarantee; (ii) a frontier-style POI heuristic that compresses the lidar field into a single goal-directed sub-target and feeds three scalar features to the policy; (iii) an empirical study of training dynamics and cross-layout generalization on consumer hardware; and (iv) a reusable, publication-quality visualisation module for the environment. All code follows a reproducible `src`-layout package with a strict commit-time quality gate.

2 Problem Formulation

2.1 Environment

The world is an 800×600 bounded field containing axis-aligned rectangular obstacles and a fixed goal near the bottom-right corner (Figure 1). At the start of *every* episode the environment samples a fresh task (Figure 2):

- the **obstacle count** is drawn uniformly from $\{2, \ldots, 5\}$;
- each obstacle's **width and height** are drawn uniformly from $[60, 220]$ pixels, its position uniformly over the field, rejecting overlaps between obstacles and placements that crowd the goal;
- the **start pose** — position and heading $\alpha \sim \mathcal{U}[0, 2\pi)$ — is drawn uniformly over the free space at least 300 pixels from the goal.

Randomization alone could produce impossible tasks (a wall of obstacles sealing off the goal). To rule these out, the sampler rasterizes the layout onto a 20-pixel occupancy grid, inflates every obstacle by the robot footprint plus a safety gap, and flood-fills the free cells from the goal by breadth-first search. Start positions are drawn only from cells connected to the goal, so *every* sampled task is solvable by construction; layouts whose free space leaves no valid start cell are resampled.

The robot carries a simulated lidar with seven beams spread uniformly over $[-90°, +90°]$ relative to its heading; each beam returns the distance to the nearest obstacle or wall, analytically computed by ray–rectangle intersection and normalised by the field diagonal $L = \sqrt{800^2 + 600^2}$.

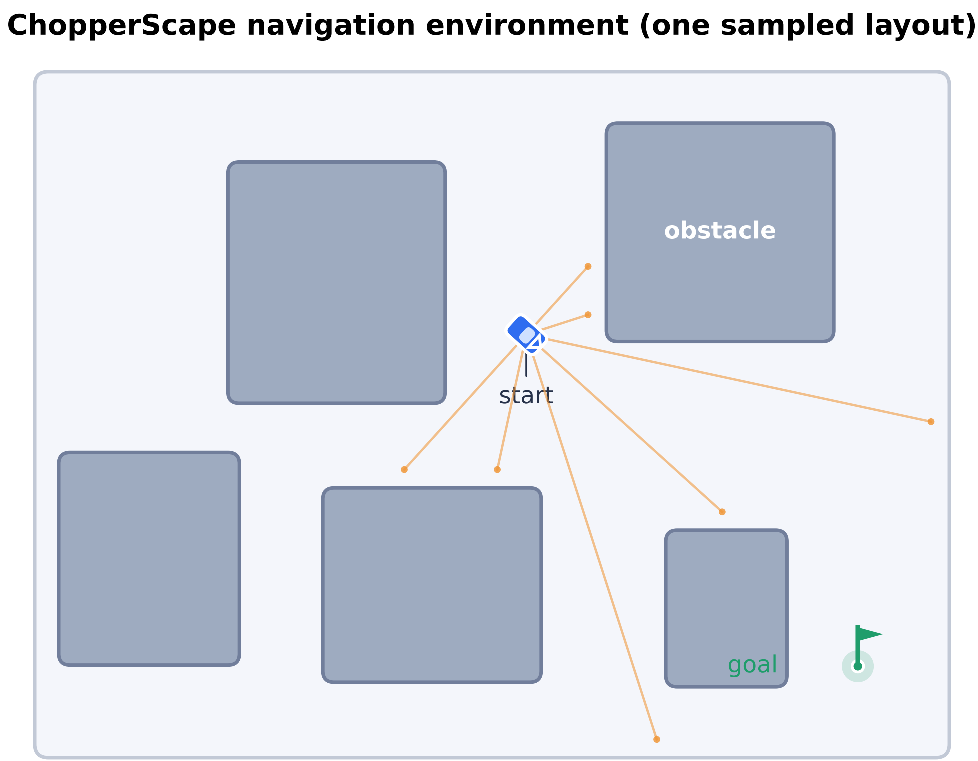

Figure 1: The *ChopperScape* environment (one sampled layout). The rover (top-down) emits seven lidar beams (orange) that terminate at the nearest obstacle or wall. Soft blue blocks are obstacles; the green flag with a tolerance halo is the goal.

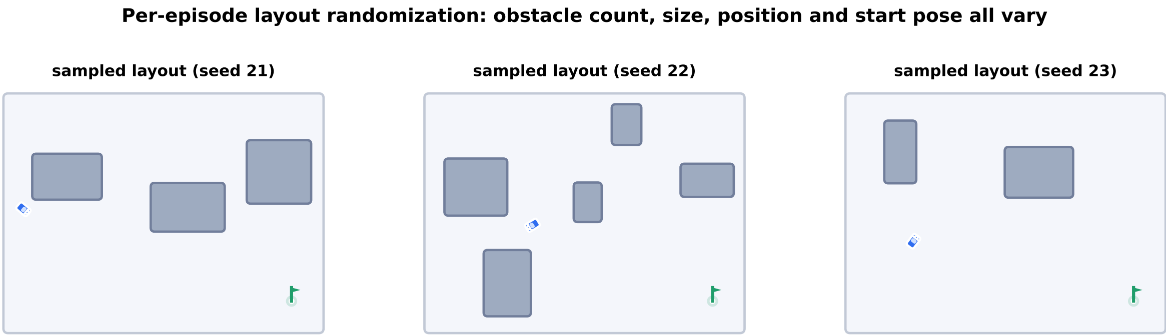

Figure 2: Three independently sampled tasks. Obstacle count, sizes, positions and the start pose all vary per episode; a BFS over the inflated occupancy grid guarantees a collision-free corridor from every start to the goal.

2.2 Markov decision process

State $s \in \mathbb{R}^{10}$. The observation concatenates the seven normalised lidar ranges $d_1, \dots, d_7 \in [0, 1]$ with three goal-directed features derived from the currently selected POI (Section 3.2):

$$s = \left(d_1, \dots, d_7, \tfrac{\theta_{\mathrm{rel}}}{2\pi}, \tfrac{\Delta\phi + \pi}{2\pi}, \tfrac{\rho}{L} \right), \tag{1}$$

where θ_{rel} is the bearing to the POI, $\Delta\phi \in [-\pi, \pi)$ the *signed* heading error, and ρ the Euclidean distance to the POI. The sign of $\Delta\phi$ matters: since the robot's own heading is not part of the observation, an unsigned error would leave the steering direction unidentifiable — the policy would know how far it points away from the sub-goal but not which way to turn. On a fixed map this ambiguity can be compensated by memorizing the route; under layout randomization it must be resolved in the state itself.

Action $a \in [-1, 1]^2$. The actor emits a throttle a_0 and a steering increment a_1 through a tanh output. They drive the kinematics

$$\alpha \leftarrow (\alpha + a_1) \bmod 2\pi, \qquad v = 10 \cdot \tfrac{a_0 + 1}{2} \in [0, 10], \tag{2}$$

$$x \leftarrow x + v \cos\alpha, \qquad y \leftarrow y + v \sin\alpha. \tag{3}$$

Reward. The agent receives a dense shaping reward that rewards forward speed and progress toward the goal and penalises turning and time, with large terminal bonuses/penalties:

$$r = \begin{cases} +101 & \text{goal reached } (\rho_g < 10), \\ -50 & \text{collision or out of bounds}, \\ a_0 - |a_1| + \kappa\,(\rho_g^{t-1} - \rho_g^t) - 1 & \text{otherwise}, \end{cases} \tag{4}$$

with $\kappa = 0.1$. The progress term is potential-based ($\Phi(s) = -\kappa\rho_g$), so it densifies the learning signal without changing the optimal policy — important under layout randomization, where the sparse arrival bonus alone is rarely reached by exploration on a map the agent has never seen. Episodes terminate on arrival, collision, leaving the field, or after 500 steps. Arrival is treated as a terminal transition in the learning target (no bootstrapping past the goal).

3 Method

3.1 Deep Deterministic Policy Gradient

DDPG [1] is an off-policy actor–critic algorithm for continuous actions. A deterministic actor $\mu_\theta(s)$ proposes an action and a critic $Q_\psi(s, a)$ estimates its value. The critic minimises the one-step temporal-difference error against slowly-tracking target networks $\mu_{\theta'}, Q_{\psi'}$:

$$y_t = r_t + \gamma\,(1 - \text{done})\,Q_{\psi'}(s_{t+1}, \mu_{\theta'}(s_{t+1})), \tag{5}$$
$$\mathcal{L}(\psi) = \tfrac{1}{N}\textstyle\sum_t \text{Huber}(Q_\psi(s_t, a_t) - y_t), \tag{6}$$

while the actor ascends the critic's value estimate, $\nabla_\theta J = \frac{1}{N}\sum_t \nabla_a Q_\psi(s,a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)$. Target networks are updated by Polyak averaging $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$. Transitions are stored in a replay buffer and sampled uniformly to decorrelate updates [3]. Because layouts change per episode, the buffer naturally mixes transitions from hundreds of distinct maps, which further decorrelates the batches. Both networks are three-layer perceptrons (Figure 3).

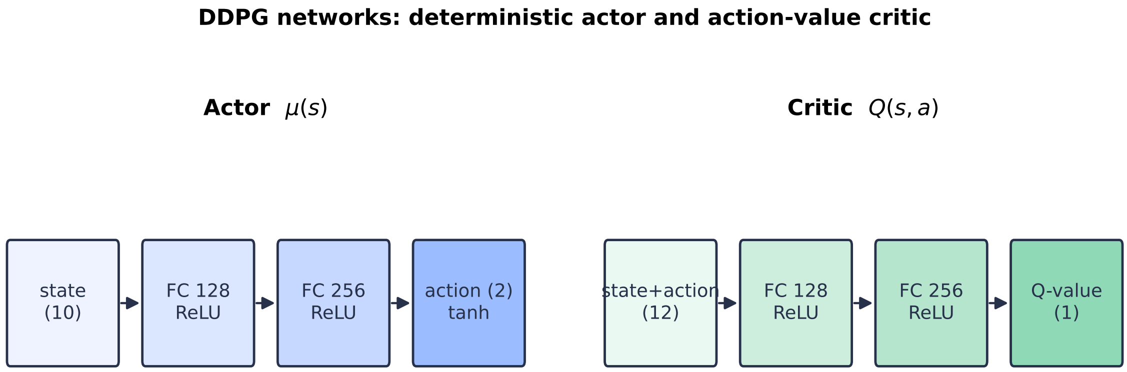

Figure 3: The two DDPG networks. The actor maps a 10-d state to a 2-d tanh action; the critic scores a state–action pair (12 inputs) with a scalar Q-value.

Exploration. Because the policy is deterministic, exploration is injected with temporally correlated Ornstein–Uhlenbeck (OU) noise [5], $\mathrm{d}x = \theta(\mu - x)\,\mathrm{d}t + \sigma\,\mathrm{d}W$, which produces smooth, momentum-like perturbations well suited to inertial control. The noise scale is annealed as $\eta_e = \max(0.1,\, 1 - e/E_{\text{noise}})$ over training episode e, with $E_{\text{noise}} = 2400$.

3.2 Frontier-style POI exploration heuristic

Feeding seven raw lidar values to the policy gives only myopic, reactive information. Inspired by frontier-based exploration [6], we instead distil the lidar field into a single *point of interest*. Along each free beam we spawn candidate frontier points and score every candidate p with

$$h(p) = \underbrace{\tanh\!\left(\frac{\exp((d_o/\ell_1)^2)}{\exp((\ell_2/\ell_1)^2)}\right)\ell_2}_{\text{reach / clearance } d_1} + \underbrace{d_g}_{\text{goal distance}} + \underbrace{\exp(\text{info}/\text{area})}_{\text{information } I}$$

with $\ell_1 = 5, \ell_2 = 20$. The agent selects the candidate of *minimum* h as its next sub-target; the three POI features above then make the global goal reachable through a sequence of locally sensible waypoints. Because the layout is new in every episode, the POI memory (the visited-space field and the candidate list) is rebuilt from scratch on each reset. Figure 4 visualises h over the free space of one layout: the basin of low score flows around the obstacles toward the goal.

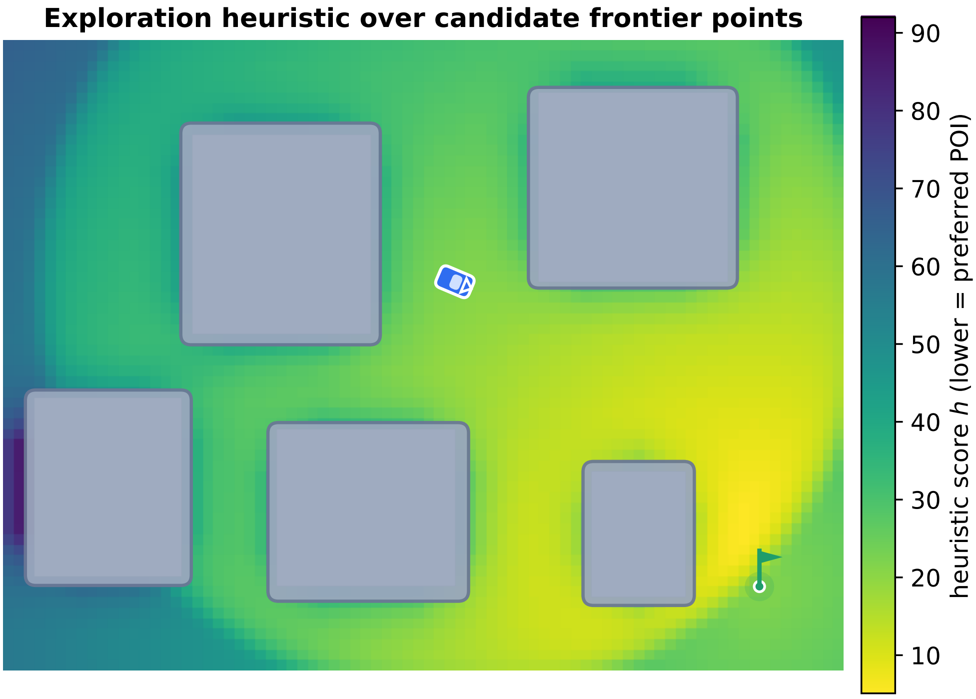

Figure 4: Exploration heuristic h evaluated over candidate frontier points for a rover at the start of one sampled layout. Lower (yellow) is more attractive; the field guides sub-target selection around obstacles toward the goal.

4 Implementation

The system is implemented in PyTorch as an installable `src`-layout package (`mobile_robot_navigation`) with Hydra-based configuration, a pure-logic training core, and a thin command-line entry point. The layout randomization is part of the typed environment config (obstacle count and size ranges, minimum start distance), so sweeps over task difficulty are one-line Hydra overrides. Device selection prefers CUDA, then Apple MPS, then CPU, so the same code runs unchanged on a laptop GPU. Quality is enforced at commit time (Ruff, `mypy -strict`, property tests, custom style checks); property-based tests assert the solvability guarantee itself, re-deriving the BFS corridor check on independently sampled layouts. Table 1 lists the hyperparameters.

Table 1: Training hyperparameters.

Parameter	Value	Parameter	Value
Episodes	3000	Replay capacity	100,000
Max steps / episode	500	Batch size	256
Discount γ	0.99	Actor LR (Adam)	1×10^{-4}
Soft-update τ	0.005	Critic LR (Adam)	5×10^{-4}
OU (θ, σ)	$(0.15, 0.2)$	Critic loss	Huber
Noise episodes	2400	Grad-norm clip	1.0
Hidden units	$128, 256$	LR schedule	linear $\to$ 0.05
Obstacles / episode	2–5	Obstacle size	60–220 px

5 Experiments and Results

Setup. We train for 3000 episodes with a fixed seed on an Apple-Silicon CPU[1]; every episode runs on a freshly sampled layout. Until the replay buffer holds at least one batch, actions are sampled uniformly; thereafter the actor acts with annealed OU noise. For evaluation we roll out the *deterministic* policy (no exploration noise) on 300 previously unseen random layouts and report the goal-arrival rate; during training we also track the per-episode return and count an episode as an arrival when its return exceeds 50 (only an arrival, worth $\approx +94$ net, clears this threshold).

Learning dynamics. Figure 5 shows the training curves. Thanks to the dense progress term the agent reaches the goal already in the first hundred episodes ($\sim$20% of them), and the arrival rate climbs steadily as the noise anneals, reaching 74.5% (true arrivals) over the final 200 episodes while the mean return rises from -26 to $+95$. Ablations during development underline how much the task formulation matters under randomization: with an *unsigned* heading error in the state the deterministic policy generalized to only 16.7% of unseen layouts, the signed error lifted it to 31.7%, and adding the potential-based progress term to the reward more than doubled it again.

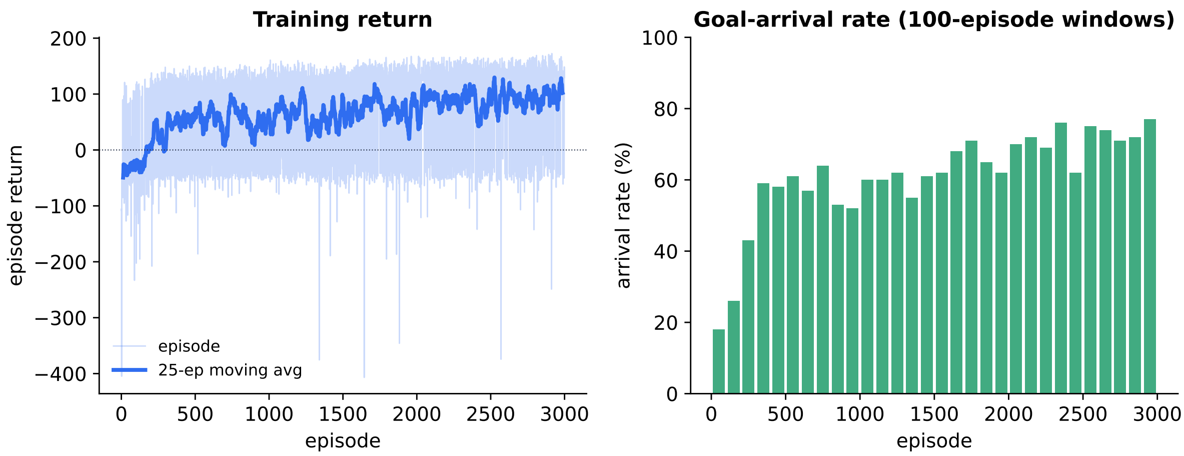

Figure 5: Left: per-episode return (faint) with a 25-episode moving average. Right: goal-arrival rate in 100-episode windows. Every episode is a new random layout.

Generalization across layouts. The headline result is the deterministic policy's arrival rate on unseen tasks: 224/300 (74.7%). Since the evaluation layouts are sampled from the same distribution but never seen during training, this directly measures whether the policy learned a navigation *strategy* — read the lidar, chase the POI sub-goal, avoid what the beams report — rather than a memorized route. Figure 6 shows a successful rollout and a characteristic failure on two unseen layouts.

[1]The networks are small enough that the CPU outpaces the MPS backend, whose per-step host–device synchronisation dominates the step time; we also observed a deterministic MPS command-buffer hang on long runs.

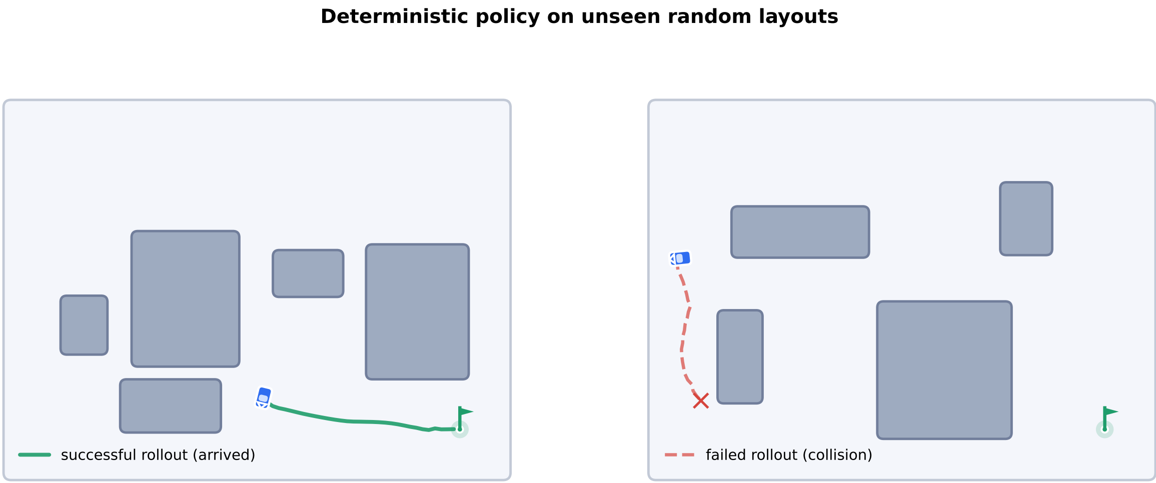

Figure 6: The deterministic policy on unseen random layouts. Left: a successful rollout into the goal halo. Right: a characteristic failure — the policy clips an obstacle while cutting a corner.

6 Visualisation

The original environment rendered a raster canvas with bitmap icons. We replaced it with a vector renderer (`viz.py`) that draws the field, rounded obstacles, a goal flag, lidar beams, trajectories, and a stylised top-down rover with heading and wheels. The same primitives back `ChopperScape.render()`, every figure in this report, and an animated GIF of policy rollouts embedded in the repository README, so the paper and the live environment are guaranteed to agree. All figures are emitted as scalable PDF.

7 Conclusion and Future Work

We presented a complete DDPG navigation agent trained under per-episode layout randomization with a BFS-certified solvability guarantee, together with a frontier-style POI exploration heuristic. On consumer hardware the deterministic policy generalizes to unseen obstacle layouts with a 74.7% arrival rate. Natural next steps: TD3-style stabilisation (twin critics, target-policy smoothing), prioritised replay, randomising the goal position as well, curriculum over obstacle density, and a learned (rather than hand-designed) sub-goal proposer.

References

[1] T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al., “Continuous control with deep reinforcement learning,” in *ICLR*, 2016.

[2] D. Silver, G. Lever, N. Heess, et al., “Deterministic policy gradient algorithms,” in *ICML*, 2014.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] G. E. Uhlenbeck and L. S. Ornstein, “On the theory of the Brownian motion,” *Physical Review*, vol. 36, no. 5, pp. 823–841, 1930.

[6] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *IEEE Int. Symp. on Computational Intelligence in Robotics and Automation*, 1997.